F1 notebook review: what was found, and how to fix it

Team 10 · F1 podium prediction, Milestone 1 · **Part 1:** my tasks as Member D (T19, T22–T26, T30) · **Part 2:** the whole notebook (members A, B, C) · checked against `main` at `a2f96ee`; first pass 14 Oct, every finding re-checked against the brief 15 Oct 2026 · deadline 18 Oct 2026, 23:59

> **How to read this version.** Every finding was checked a second time against the notebook, DECISIONS.md, the figures and the milestone brief (*MS1 Description*). Each one now has a priority:
>
> - **REQUIRED** the brief asks for it, or the text says something the notebook's own output contradicts. Do these before submitting.
> - **RECOMMENDED** not in the brief, but cheap, and it answers a question an evaluator is likely to ask.
> - **OPTIONAL** an item from the team plan or the dataset notes, or polish. The brief is met without it. A few are marked "skip".
>
> Of the 53 findings, **12 are required, 19 recommended and 22 optional**. All 12 required items can be done as text edits, so none of them needs a new run. Four findings were corrected in this version: NT-3 (its fix would have introduced an error), C-1 (its evidence was misread), NW-2 (the sentence does follow from the output) and NC-4 (the pin would not control Kaggle's versions). The notebook has already been run and checked on Kaggle.

Required before submission (≈ 40 min, text only)

#	where	owner	what to change	effort
S-3	§9.4 calibration panel; D20 (§9 callout, §9.5, DECISIONS)	D	D20's evidence plot is drawn on test. Draw it on validation, or (text only) cite the validation Brier scores as the evidence and call the plot "for reference".	5–10 min
S-4	DECISIONS D15	D	Delete the "Empirical check (§8.4)" paragraph: that trial is no longer in the notebook.	5 min
T-1	§11.3 "How to read it"	D	The contributions add +5.01; the total is +1.47 log-odds.	1 min
T-3	§8 D19 callout + comment	D	Qualifying is missing before 1994 and patchy until 2002, not "before 2003".	2 min
T-4	§7 "What this shows"	D	Pre-1994 seasons also feed `experience`, the form features and §8.4.	2 min
T-6	§9.4 `MAIN`, or D21	D	D21 says log-loss is reported; the table does not show it.	2 min
T-7	§6.3 D15 callout + DECISIONS D15	D	Race-fuel qualifying ran 2006–2009, not 2006–2008.	1 min
T-9	DECISIONS D23, D24, D25	D	Tags and evidence pointers that do not match the notebook.	5 min
NS-7	§6.4 + DECISIONS D13	C	The season-opener test D13 quotes is not in the notebook: say it was measured in an earlier version.	5 min
NS-8	DECISIONS D17	D17's owner	"§8.5" → "§8.4" (two places).	2 min
NW-1	§10 "What this shows"	C	For gradient boosting, removing grid costs most, not standings and qualifying.	2 min
NW-3	§8.2 E2 + DECISIONS D19	B	Test scores are listed as support for a choice: mark them "for reference only".	2 min

The short version. My part meets the brief. Every model has all five metrics on train, validation and test, and every choice (thresholds, settings, early stopping, which model to use) was made on validation, never on test. Nothing below changes a result or the chosen model. The findings are of three kinds: **gaps against the brief and the plan** (S-1 to S-6), **sentences that say something slightly different from the code** (T-1 to T-9), and **small code and figure fixes** (C-1 to C-9). Eight of them are required, and all eight can be fixed in the text.

8	7	9
required	recommended	optional or skip
(S-3, S-4, T-1, T-3, T-4, T-6, T-7, T-9)	(S-1, S-2, T-2, T-5, T-8, C-3, C-4)	(S-5, S-6, C-1, C-2, C-5 to C-9)

1. Gaps against the brief and the plan

S-1 RECOMMENDED **"Reached Q3" was never built, and nothing says why**

Found T19 asks for "gap to pole %, **reached Q3**, grid – quali position, normalised grid, field size, home flag, driver age". All are built except "reached Q3". The dataset notes (§5, §9) also recommend it. D15 and the notebook never mention it.

Why it matters The brief leaves the feature list to us ("design room"), so the feature itself is not required. But an evaluator who compares our features with the plan or the notes sees one dropped without a reason, and one line of reasoning closes that.

Fix Do not build it now: it would change every number after §6. Add one bullet to D15 (in DECISIONS.md and in the §6.3 callout): "*No 'reached Q3' flag: from 2006, Q3 is exactly the ten fastest qualifiers, so the flag equals* `quali_position ≤ 10`*, which the model already has; before 2006 it does not exist, so it would add one more column that is missing by era.*"

Optional evidence cell after §6.3, if you are re-running anyway (expect slightly under 100%: a driver who reached Q3 but set no time has no `q3_sec`):

```
# Is "reached Q3" a new piece of information? From 2006 Q3 is the top-10
shoot-out.
q06 = df["qualifying"].merge(races[["raceId", "year"]],
on="raceId").query("year >= 2006")
same = q06["q3_sec"].notna() == (pd.to_numeric(q06["position"]) <= 10)
print(f"'reached Q3' equals 'qualified in the top 10' for {same.mean():.1%} of
2006–2024 qualifying rows")
```

Effort 5 min (+10 min for the optional cell)

S-2 RECOMMENDED Class weights were compared for the random forest only

Found

D20 says weights "ruin" calibration, and shows it by comparing the *weighted forest* (Brier 0.106) with the *unweighted logistic regression* (Brier 0.067). The logistic regression, gradient boosting and the network were never trained with weights.

Why it matters

That comparison mixes two things: the model type and the weights. An evaluator can say "the forest is badly calibrated because it is a forest, not because of the weights". The brief does not ask for class weights (T24 in the plan does), but this is an easy question to ask, and the cell below answers it.

Fix

Add one cell in §9 (after the baseline): the **same** model with and without weights, on validation.

```
# D20 evidence: the same logistic regression with and without class weights
(validation only)
cw_rows = []
for cw in [None, "balanced"]:
    m = LogisticRegression(max_iter=3000, class_weight=cw).fit(X_train_scaled,
y_train)
    p = m.predict_proba(X_val_scaled)[:, 1]
    cw_rows.append({"class_weight": str(cw), **metrics(y_val, p,
best_f1_threshold(y_val, p))})
display(pd.DataFrame(cw_rows).round(4))
```

Published results on logistic regression predict about the same PR-AUC and ROC-AUC, with a clearly worse Brier score (van den Goorbergh et al., 2022). That would support D20 directly. If the cell shows something else, update D20.

Effort

20 min + Run All

S-3 REQUIRED The calibration plot used as evidence is drawn on test

Found

§9.4 draws calibration with `calibration_curve(y_test, …)` and titles it "on test", and D20 (§9 callout, §9.5 and DECISIONS.md) cites that plot as its evidence.

Why it matters

The brief: thresholds and hyperparameters are chosen "using training and validation data only – never the test set". Class weights are a setting of the forest, and team rule 6 says the same. D20 itself says an earlier test-based justification was removed.

Fix

Either (code, same cell as C-3) draw the calibration panel on validation: `calibration_curve(y_val, RESULTS[name]["p"]["validation"], …)`, and retitle it "Calibration on validation". **Or** (text only, no re-run) make D20 cite the validation Brier scores as its evidence (weighted forest 0.106, unweighted logistic regression 0.067, both in the validation table), and call the plot "drawn on test, for reference only".

Effort

10 min (code) or 5 min (text)

S-4 REQUIRED D15 cites an experiment that was removed

Found D15 is tagged "[DOMAIN, tested in §8.4]" and quotes a raw-lap-time trial (0.7255 vs 0.7233). That trial was deleted from the notebook; §8.4 is now the training-window check.

Why it matters The brief: when a choice is justified empirically, "you need to keep these trials in your notebook for verification". An evaluator who opens §8.4 finds something else.

Fix In DECISIONS.md D15:

- delete the paragraph that starts "**Empirical check (§8.4).**";
- change the tag to **[DOMAIN]**;
- change "Evidence. §6.3 …, §6.7, §8.4." to "Evidence. §6.3 …, §6.7."

The domain reason (lap length differs by circuit) stands on its own.

Effort 5 min

S-5 OPTIONAL The grid baseline is "grid ≤ 3" only because the search landed there

Found The rule scores `1/grid`, and its threshold comes from the same max-F1 search as the models. It landed on 0.333, which is exactly grid ≤ 3, but nothing in the code or the text says so.

Why it matters T23 and the notes define the baseline as "podium if grid ≤ 3". The table already prints 3.000 predicted podiums per race, so this is only about making it obvious.

Fix Add one sentence to the grid-rule row of the §9 table: "*The threshold chosen on validation is 0.333, i.e. exactly 'podium if grid ≤ 3' (3 predicted podiums per race).*"

Effort 2 min

S-6 OPTIONAL D19 overstates the Pipeline

Found D19 says preprocessing is "fitted on the training set inside a `Pipeline`". In §8–9, the imputer, the scaler and the models are fitted step by step and wrapped in a Pipeline afterwards.

Why it matters The behaviour is identical (§8.4 asserts it), but the sentence says something the code does not do.

Fix Reword D19: "*… all fitted on the training set only; the fitted steps are then wrapped in a `Pipeline` for reuse, and §8.4 asserts it gives identical predictions to the step-by-step version.*"

Effort 2 min

2. Sentences to correct

These say something different from what the code or the outputs show. Each is a find-and-replace. T-1 to T-5, T-7 and T-8 are in the notebook markdown or comments, T-6 is code (with a text-only alternative), and T-9 is in DECISIONS.md. T-1, T-3, T-4, T-6, T-7 and T-9 are required: the notebook's own output or the record contradicts them.

#	priority	where	now	replace with
T-1	**required**	§11.3 "How to read it", first bullet	"the contributions add up to **+1.47** log-odds"	"the contributions add **+5.01**, bringing the total to **+1.47** log-odds" (–3.535 + 5.007 = 1.472)
T-2	**recommended**	§11.3 code comment above the what-if	`# … a different grid slot and no penalty.`	`# … a different grid slot; grid_pct and grid_penalty recomputed for it.` (The code gives penalties 0, 5 and 9.)
T-3	**required**	§8 D19 callout + the comment in the next cell	"no qualifying recorded before 2003"	"no qualifying recorded before 1994, and patchy until 2002". The notebook's own output: 53.6% of 1994–2002 rows are missing it.
T-4	**required**	§7 "What this shows"	"Seasons before 1994 are used only for the EDA and the data-engineering questions."	"Seasons before 1994 are not used to fit the models; they are used for the EDA, the DE questions, the history behind `experience` and the form features, and the window check in 8.4."
T-5	**recommended**	§9.5, "The network" bullet	"(validation 0.727 averaged over 3 seeds, against 0.723; test 0.705 vs 0.709)"	"(validation 0.727 and test 0.701, both averaged over 3 seeds, against 0.723 and 0.709)". 0.705 is seed 42 alone; not wrong, but it mixes two bases in one sentence.
T-6	**required**	§9.4 code, `MAIN = [...]`, or D21	D21 says log-loss is reported, but the table does not show it	add `"log-loss"` after `"Brier"` in `MAIN`; or, text only, say in D21 that log-loss is computed for every model and saved in `outputs/model_comparison.csv`
T-7	**required**	§6.3 D15 callout + DECISIONS D15	"in 2006–2008 the final session was … run with race fuel"	"2006–2009". Refuelling ended in 2010; the dataset notes §3.2 say 1994–2009.
T-8	**recommended**	§9.1 code comment; §9.6 table	"memorised its training rows"; "far above its out-of-bag score"	"is inflated because each tree has seen most of its training rows"; "above its out-of-bag score (0.695 vs 0.648)". With 50 rows per leaf, the forest cannot memorise single rows.
T-9	**required**	DECISIONS.md tags and pointers	D24 [EXP]; D25 [EXP] "(model summary, …)"; D23 "used in §8.4, §9.4, §10"	D24 **[DOMAIN, checked by EXP]** (it keeps the lower scorer by rule); D25 **[DOMAIN]**, " (weight count, learning curves, seed table)"; D23 "used in §9.4 and §10". Tell D17's owner that "§8.5" should be "§8.4" (NS-8).

3. Small code and figure fixes

C-1 OPTIONAL · SKIP **TensorFlow op determinism is not switched on**

Found: §8.2 sets `keras.utils.set_random_seed(SEED)`, but never calls `tf.config.experimental.enable_op_determinism()`. **Correction to the first version of this review:** the "slightly different numerical results" line in the saved output is TensorFlow's standard oneDNN start-up message, printed on every CPU start. It is not a sign that runs differ, and within the saved run the two seed-42 networks agree (0.7290). **Why skip:** turning determinism on can change every network number quoted in the markdown and DECISIONS.md (E2, the seed table, §9.5, D19, D25), and the notebook has already been run and checked on Kaggle. The README and §1 already say the network can move slightly under other library versions. **If asked:** the seed is fixed, and the 3-seed table shows how much the network moves (validation 0.726–0.729).

C-2 OPTIONAL **A conclusion is printed before it is computed**

Found: the §9.1 forest cell prints "a gap of 0.002, *inside the forest's seed spread (9.3)*, so that part of the choice is noise". The seed spread is only computed later, in 9.3. **Why:** it is true today (0.002 vs a 0.005 spread), but after a run under other library versions the notebook could print a sentence that is no longer true. **Fix:** print only the two numbers, and write the conclusion in the 9.3 markdown, next to the seed table.

C-3 RECOMMENDED **Fig 19: the calibration panel uses different colours**

Found (confirmed in fig 19): the calibration panel skips the grid rule, so matplotlib's colour cycle shifts by one. Orange is the logistic regression in the ROC panel but the random forest in the calibration panel. **Why:** a reader can attribute the badly calibrated orange line to our chosen model; only the line width tells them apart. **Fix:** give each model one fixed colour, and pass `color=COLORS[name]` in every plot loop of §9.4.

```
COLORS = dict(zip(RESULTS, plt.rcParams["axes.prop_cycle"].by_key()["color"]))
# e.g. axes[1].plot(mean_pred, frac, marker="o", color=COLORS[name], lw=2.5 if name ==
FINAL_MODEL_NAME else 1, label=name)
```

C-4 RECOMMENDED **The SHAP waterfalls show scaled values**

Found (confirmed in fig 24): the waterfall reads "5.648 = drv_win_share_before", where the real value is 0.82, because SHAP labels the bars with the inputs the model sees, which are scaled. Four bar labels are also clipped: "+0.69" looks like "−0.6". **Why:** the brief asks for local explanations "plotted and interpreted", and this is the figure most likely to be shown in the evaluation. **Fix:** add one line before the waterfall plots in §11.3. The SHAP numbers do not change, only the labels. If labels still clip, make the figure wider.

```
shap_values.display_data = FINAL_PIPE.named_steps["impute"].transform(X_test)   # unscaled
values for the labels
```

C-5 OPTIONAL **Explain why the network's train curve is below validation**

Found: §9.3 explains "validation above train" only by era difficulty. **Why:** for the NN curves there is a second, Keras-specific reason, and it is a likely question. **Fix:** add one sentence: "*In the curves, Keras computes the training metric with dropout switched on and averages it over the epoch while the weights change (Keras FAQ); measured afterwards without dropout, training PR-AUC is 0.681.*"

C-6 OPTIONAL **Tuned gradient boosting is under-confident: say so**

Found (fig 19): its top calibration bin predicts about 0.46 where about 0.68 of drivers podium. §9.5 discusses calibration only for the forest. It is not the inference model, so this is a completeness point. **Fix:** add one sentence to §9.5: a slow learning rate (0.01) with only 100 trees keeps its probabilities close to the base rate.

C-7 OPTIONAL **Missing-value indicator weights: one sentence**

Found: the logistic regression gives the three qualifying "was missing" columns large weights of opposite sign: +0.71, −0.32, −0.32. They cancel when all three are missing, the usual pattern in the 1990s. A test row whose only missing qualifying value is the Q1 time is pushed up by about +1.5 log-odds (computed from the weight, visible in the beeswarm). §11.2 already calls such weights artefacts of correlated inputs. **Fix:** one sentence in §11.2.

C-8 OPTIONAL **One cell crashes if it is re-run on its own**

Found: in §9.4, `scores = pd.concat(scores, …)` turns the list of tables into one table under the same name. Run All works, but **re-running that cell alone raises a `TypeError`**. The brief asks for Run All, which works. **Fix:** give the table its own name, e.g. `comparison = pd.concat(scores, …)`, and use `comparison` in that cell and in the plot cell after it. Simplest: in the evaluation, use Restart & Run All (see NC-3).

C-9 OPTIONAL **Tidy-ups (do the one-liners, skip the rest)**

- `import tensorflow as tf` is unused (C-1 is skipped); delete it.
- `explain_row` labels an unknown column "?" instead of failing. Every column has a group, so drop the `"?"` default.
- The per-period table in §9.4 hard-codes the split years (`2019, 2021`); use `TRAIN_END` and `VAL_END`.
- SHAP prints a warning for `max_samples=1000`. **Leave it:** using the full training set would change the base value and every SHAP number quoted in §11 slightly.
- The random forest, gradient boosting and logistic regression are built by hand in places although `make_rf`, `make_hgb` and `make_lr` exist. **Leave this** until after the deadline: a refactor now risks new bugs for no change in results.

NO CHANGE **Test scores are printed in every model cell**

Team rule 6 says "look at test metrics once, at the end", but every model cell prints test next to validation. I checked that **no choice reads them**: thresholds, both search grids, early stopping, model selection and the tree pick for the ablation all use validation. The brief asks for test performance to be reported, so printing it is fine. If asked, say exactly that.

4. Order of work

□ **Required, text only, no re-run** (≈ 25 min): S-3 (text route), S-4, T-1, T-3, T-4, T-6 (text route), T-7, T-9.

□ **Recommended, text only** (≈ 15 min): S-1 (D15 bullet), T-2, T-5, T-8.

□ **Recommended, code** (≈ 1 h, only if there is time for a new run): S-2 class-weight cell, C-3 colours (with the code route of S-3 in the same cell), C-4 waterfall labels.

□ **If any code changes:** Run All on Kaggle once, then check that every number quoted in the markdown and in DECISIONS.md still matches the new outputs.

□ **Optional, or after the deadline:** S-5, S-6, C-2, C-5 to C-9. **Skip:** C-1.

PART 2

The whole notebook: what the review found, and how to fix it

Members A, B and C's sections (everything except Part 1): audit, cleaning, EDA, the three data-engineering questions, features, preprocessing experiments, ablation, permutation importance, LIME, inference, summary, README. Owner tags follow `Team_plan.pdf`: **A** data lead, **B** data-engineering analyst, **C** feature engineer.

> **The short version.** The rest of the notebook is also correct where it matters most. No feature leaks: races are ordered by date, standings come from the previous race (asserted), form uses `shift(1)`, and team form is collapsed per race first. No join can duplicate or drop rows: every parent key is unique and the merges use `validate=`. The DE answers follow from their printed numbers. The numbers quoted in the markdown match their outputs for Q1–Q3, E1–E3, the window check, the ablation and the §13 summary.
>
> What is left falls into three groups:
>
> - **Plan items that were never built**, or built only in part: a profile per table, the raw missingness-per-season heatmap, the lineage map, three leakage asserts. None of these is required by the brief.
> - **Two statements that do not follow from the outputs** (NW-1, NW-3), and one method point (NW-2).
> - **Text and code fixes** like those in Part 1.
>
> Four items here are required (NS-7, NS-8, NW-1, NW-3), and all four are text edits. The earlier review's open items (S2–S5, N1–N10) are all **still open**; they are included below.

4
required
(NS-7, NS-8, NW-1, NW-3)

12
recommended
(NS-3, NS-4, NS-5, NS-9, 8 NT text fixes)

13
optional or skip
(NS-1, 2, 6, 10, NW-2, NT-3, NT-8, NC-1 to NC-6)

5. Gaps against the brief and the plan

NS-1 OPTIONAL **No raw "% missing per column per season" heatmap, and no before/after missingness**

owner C (T06) · A (T10)

Found

The only per-season heatmap (§6.7, fig 13b) shows the *cleaned model features* from 1994. The raw audit (§2.1) shows missingness per column with no season axis, and §4 has before/after plots for rows and base rate but not for missingness. T06 asks for "% missing per column per season heatmap … q1/q2/q3 coverage". T10 asks for "before/after plots for row counts, missingness, base rate". The dataset notes (§5) call it "your key EDA plot".

Why it matters

The brief asks for EDA before and after cleaning, but not for this exact plot, and the biggest missingness drift (qualifying) is already shown per season in §2.3 and fig 13b. It would make the before/after story complete.

Fix

Add one cell at the end of §2.3 (raw). Repeat it on `results_clean` in §4 for the "after" view.

```
# RAW: % missing ('\N' or '') per column per season, results and qualifying
(the notes' key EDA plot)
yr = raw["races"].set_index("raceId")["year"].astype(int)
fig, axes = plt.subplots(2, 1, figsize=(16, 9))
for ax, t in zip(axes, ["results", "qualifying"]):
    miss = raw[t].isin([SENT, ""]).groupby(raw[t]
["raceId"].map(yr)).mean().mul(100)
    miss = miss.loc[:, miss.max() > 0]
    sns.heatmap(miss.T, cmap="Reds", vmin=0, vmax=100, ax=ax, cbar_kws=
{"label": "% missing"})
    ax.set_title(f"RAW {t}: % missing per column per season");
ax.set_xlabel("season")
save_fig("03b_raw_missing_per_season")
```

Effort

20 min · new cell, then Run All

NS-2 OPTIONAL **No profile table per CSV** owner A (T03)

Found T03: "Profile function (Lab 3 'six questions': dtype, nulls, distinct, min/max, top values) run on all 14 tables · Done when: one profile table per CSV in the notebook". The notebook only prints rows × columns per table (§1). Its audit checks are targeted (sentinels, eras, coverage, spelling, time-like text), not a profile.

Why it matters A plan item, not a brief requirement: the audit in §2–3 already covers every table (placeholders per column, primary and foreign keys, time-like text). If you add it, show one combined table rather than 14, so the notebook stays readable.

Fix Add one cell at the start of §2:

```
def profile(d):
    """Lab 3's six questions per column: type, missing, distinct, min, max,
most common value."""
    rows = {}
    for c in d.columns:
        s = d[c].replace({SENT: np.nan, "": np.nan})
        num = pd.to_numeric(s, errors="coerce")
        is_num = num.notna().sum() == s.notna().sum()
        rows[c] = {"looks like": "number" if is_num else "text", "missing":
int(s.isna().sum()),
                   "distinct": int(s.nunique()), "min": num.min() if is_num
else None,
                   "max": num.max() if is_num else None, "top value":
s.mode().iat[0] if s.notna().any() else None}
    return pd.DataFrame(rows).T
display(pd.concat({t: profile(raw[t]) for t in TABLES}, names=["table",
"column"]))
```

Effort 15 min

NS-3 RECOMMENDED **No constructor lineage map, and Q3's "no stable unreliable team" depends on that** owner B (T13, T16)

Found The word "lineage" appears nowhere in the notebook or DECISIONS.md. T13 asks for a hand-written lineage dict with an assert, and T16 for Q3 "by lineage". The notes (§3.4) say reporting by lineage "is usually more meaningful here". D11 does say that Q3 groups by `constructorId`, so the choice is documented. But the conclusion (ρ = –0.49 over 11 teams present in both periods) compares like-named entries only: Alpine's and Aston Martin's 44 starts from 2021 alone are paired with 2022–24, while Renault, Racing Point and Toro Rosso drop out of the comparison.

Why it matters The answer changes with the grouping. Recomputed by hand from the printed Q3 tables, by lineage 10 teams are in both periods and the rank correlation is about **+0.27**, against –0.49 by `constructorId`. Renault → Alpine is the second-highest lineage in 2014–2021 (13.2%) and the highest in 2022–2024 (10.4%): a stable unreliable team. An evaluator who knows Toro Rosso became AlphaTauri, then RB, will ask about it.

Fix **Text only (≈ 5 min):** soften the §5.3 sentence to "by `constructorId` there is no stable unreliable team; renamed teams are counted separately, and as one lineage Renault → Alpine is high in both periods (13.1% and 10.4%)". **Better (code):** add a lineage view as a robustness table in Q3, keeping the per-`constructorId` answer, and confirm the numbers above before quoting them.

```
# Hand-written lineage (notes §3.4): name -> current team; names not listed
are their own lineage.
LINEAGE = {"Toro Rosso": "RB", "AlphaTauri": "RB", "RB F1 Team": "RB",
           "Force India": "Aston Martin", "Racing Point": "Aston Martin",
           "Renault": "Alpine", "Lotus F1": "Alpine", "Alpine F1 Team":
"Alpine",
           "Alfa Romeo": "Sauber", "Marussia": "Manor", "Manor Marussia":
"Manor"}
assert set(LINEAGE) <= set(q3["constructor"]), "a lineage name does not match
the data"
q3["lineage"] = q3["constructor"].map(LINEAGE).fillna(q3["constructor"])
lin = q3.groupby(["period", "lineage"])["mech"].agg(starts="size", mech="sum")
lin["rate"] = lin["mech"] / lin["starts"]
display(lin[lin["starts"] >= 30]["rate"].unstack("period").round(3))
```

Check `q3["constructor"].unique()` once for any other renamed entry, and add it.

Effort 5 min (text) or 30 min (code)

NS-4 RECOMMENDED **The nationality map does not fail loudly** owner C (T15)

Found T15 asks for "nationality → country map (assert fully mapped)", and the notes (§8) say "unmapped values must fail loudly". §5.2 *prints* the 14 nationalities with no circuit country, and `home_flag` silently treats any unmapped value as "away". Nothing is wrong today (the data is fixed and D10 explains the rule), but a new or misspelt nationality would never be noticed.

Fix Make the "no home circuit" list explicit and assert:

```
NO_HOME_CIRCUIT = {"Chilean", "Colombian", "Czech", "Danish", "Finnish",
"Indonesian", "Irish", "Liechtensteiner",
                   "New Zealander", "Polish", "Rhodesian", "Thai",
"Uruguayan", "Venezuelan"}
unknown = set(q2["nationality"]) - set(NAT_TO_COUNTRY) - NO_HOME_CIRCUIT
assert not unknown, f"nationalities with no mapping decision:
{sorted(unknown)}"
```

Effort 5 min

NS-5 RECOMMENDED **The leakage test for form is one fixed row** owner C (T20) · earlier S5

Found T20 asks to "assert first race per driver has NaN form; recompute one feature by hand for 3 random rows". The notes (§6.3) say the same. The notebook has one hand check (Verstappen's 61st race) and no first-race assert.

Why it matters "How do you know your form features don't leak?" is the most likely evaluation question. The code is correct by reading, and the brief's checklist item ("leakage-safe form feature") is met, but one row is thin evidence.

Fix Add after the form cell (§6.5):

```
# Every driver's and every team's first race has no form history (the current
race is never in its own window).
assert base.sort_values("race_seq").groupby("driverId").head(1)
["drv_podium_rate_last5"].isna().all()
assert con_race.groupby("constructorId").head(1)
["con_podium_rate_last5"].isna().all()
# Recompute the driver feature by hand for 3 random rows.
for _, r in base.dropna(subset=["drv_podium_rate_last5"]).sample(3,
random_state=SEED).iterrows():
    prev = base[(base["driverId"] == r["driverId"]) & (base["race_seq"] <
r["race_seq"])].sort_values("race_seq").tail(5)
    assert np.isclose(prev["podium"].mean(), r["drv_podium_rate_last5"])
print("✔ first races have no form; 3 random rows recomputed by hand")
```

Effort 10 min

NS-6 OPTIONAL **Driver form and constructor form are one group, and there is no driver DNF rate** owner C (T18, T28)

Found The plan (T28) and the notes (§9) ablate "driver form" and "constructor form" as separate groups. The notebook has one `form` group, and D16 does not say why. T18 lists a driver "DNF rate over last N races", which is absent; D14 gives no reason. The brief leaves the groups and features to us, so neither is required.

Fix At minimum (2 min, no re-run), add one line to D14 saying why the driver DNF rate was left out, for example that team reliability is already covered by `con_mech_rate_last10`. Splitting the form group adds two ablation rows but changes the ablation output, so do it only with a re-run:

```
DRV_FORM = ["drv_podium_rate_last5", "drv_avg_finish_last5"]
CON_FORM = ["con_podium_rate_last5", "con_avg_finish_last5",
"con_mech_rate_last10"]
variants += [("without driver form", [f for f in FEATURES if f not in
DRV_FORM]),
             ("without constructor form", [f for f in FEATURES if f not in
CON_FORM])]
```

Effort 2 min (D14 line) · 15 min + Run All (ablation)

NS-7 REQUIRED **D13 quotes a season-opener test that no cell runs** owner C · earlier S2

Found D13 quotes "+0.0001 (95% interval −0.001 to +0.002)", a permutation importance of 0.0001 and a standalone AUC of 0.500. §6.4 says "a test with the flag in the model showed no measurable gain (D13)". No cell produces these numbers since the flag was removed, so D13 and §6.4 point at each other.

Why it matters The brief: an empirical justification needs its trials kept in the notebook "for verification". An evaluator who asks to see the test cannot be shown it.

Fix Reword both places: "measured in an earlier version of the notebook (+0.0001, CI −0.001 to +0.002); the flag was then removed". The five domain reasons in D13 stand on their own. Alternatively, restore a small cell that runs the check.

Effort 5 min

NS-8 REQUIRED **Stale evidence pointers in DECISIONS.md** earlier S3

Found D17 cites "§8.5" twice; the check is §8.4. D23 says the bootstrap is "used in §8.4"; it is not. Part 1 T-9 covers D23 and D25.

Why it matters DECISIONS.md is our sheet for answering questions in the evaluation ("open the notebook section listed under Evidence"); a pointer to a section that does not exist fails exactly then.

Fix D17: "§8.5" → "§8.4" (two places).

Effort 2 min

NS-9 RECOMMENDED **The inference function derives `grid_penalty` without the sprint rule**

owner A (T32) · earlier S4

Found `predict_podium` computes `grid_pos − quali_position`, while §6.3 uses the sprint result as the reference on 2021–22 sprint weekends. The §12 text says derived inputs use "the same formulas as Section 6".

Fix Add one sentence to §12: "For 2021–22 sprint weekends pass `grid_penalty` explicitly." No future race is affected, because qualifying sets the grid from 2023.

Effort 2 min

NS-10 OPTIONAL **Smaller plan items done only in part**

owner A (T07, T08) · B (T05, T31)

- **T05** asks for races per season, status value counts and duplicates by year as plots. There is no races-per-season plot, status appears only as category totals, and duplicates are printed per decade.
- **T07** asks for "Int64 for nullable ints; before/after dtype". There is no dtype table, and `to_numeric` gives floats.
- **T08** asks for each filter's count in decisions.md. D17 has no count for the training-window cut (12,420 rows before 1994).
- **T31** asks for LIME on "a surprise podium and a pole sitter who missed". LIME is run on one row; the miss gets SHAP only.

Each is a 5–10 minute addition. They are optional: the brief itself is met.

6. Statements to correct

NW-1 REQUIRED **§10: "Standings and qualifying matter most for gradient boosting"**

owner C

Found: the ablation output shows that removing **grid** costs gradient boosting −0.0263 on validation, more than standings (−0.0257) or qualifying (−0.0189), so the sentence contradicts the table above it. **Fix:** "For gradient boosting on validation, removing grid, standings or qualifying each costs 0.02–0.03, but all three intervals include 0, and on test only grid matters."

NW-2 OPTIONAL **§4: the penalty trend averages in 2003, a season the text itself calls an artefact**

owner A

Found: "1.0 car per race … in 2003–2013, against 1.5 in 2014–2024". **Correction to the first version of this review:** the sentence matches the printed output (1.02 vs 1.5), so it does follow from the outputs. The point is a method one: 2003 shows zero grid changes, which §4 says is probably how the data stores that season, and its zero pulls the 2003–2013 average down. Without 2003 it is about 1.1 (11 × 1.02 / 10), so the conclusion (more penalties in the hybrid era) holds either way. **Fix (text only, if wanted):** add "(about 1.1 if 2003 is left out)" to the sentence. The code change is not needed.

NW-3 REQUIRED **E2 (§8.2) and D19 use test scores to support a preprocessing choice**

owner B (T27)

Found: "For the network, impute → scale scores higher on validation (0.7290 vs 0.7204) **and on test** (0.7054 vs 0.6943)". D19 repeats it. The brief says choices are made on training and validation data only, and team rule 6 and the DECISIONS.md header say test never supports a choice. **Fix:** keep the validation numbers as the reason, and move the test numbers to "(test, for reference only: …)".

7. Text and code fixes

#	priority	where · owner	now	replace with
NT-1	recommended	§0 intro · A	"Every markdown section ends with a short 'What this shows' note"; "wherever a difference … matters, it comes with a 95% interval"	"Most sections end with …"; "the main comparisons come with a 95% interval" (§3.1, 3.4, 6.1–6.6 have no such note; E1–E3 and §8.4 have no intervals)
NT-2	recommended	§2.1 "What this shows" · A	"The 68 empty strings all sit in qualifying.q2/q3 of **2023–2024** races"	drop "of 2023–2024 races" (no output shows the years), or print them
NT-3	optional	§2.2 "What this shows" · A	"the Indianapolis 500's 33-car fields"	"the Indianapolis 500's large entry lists (35 on average, 55 in 1954)", because the plot counts entries, not starters. **Leave §3.6 and D5 as they are:** the Indy 500 grid really was 33 cars. *(Corrected: the first version also proposed changing §3.6, which would have been wrong.)*
NT-4	recommended	§4 before the save cell · A	"Save the cleaned table … for the GitHub repo"	"Save the cleaned table to outputs/" (`.gitignore` excludes `*.csv`, so it never reaches GitHub)
NT-5	recommended	§10 intro · C	"For the random forest every variant is trained with 3 seeds"	delete, or "if the random forest were the best tree model, …" (only LR and tuned GBM are ablated)
NT-6	recommended	§10 "What this shows" · C	grid-only losses "with intervals **far** below 0"	"entirely below 0" (the LR test interval ends at −0.015)
NT-7	recommended	D3 and the §3.4 comment · B	"§3.4 prints the full mapping"	"§3.4 prints every category with its label count; the full list is in `STATUS_GROUPS`". The print cuts lists of 20+ labels.
NT-8	optional	DECISIONS D4 tag	[EXP]	[EDA]: compared on integrity checks over all seasons, not on validation
NT-9	recommended	DECISIONS D6 · A	(no mention)	add: "`never_on_grid` also uses `laps`, which is post-race; it affects only rows with grid 0 and no qualifying row" *(earlier N10)*
NT-10	recommended	README · A	"the exact versions of the saved run are … printed in the notebook's first cell"; "about 10 minutes"	The first cell prints `lime unknown` and no matplotlib, seaborn, scipy or statsmodels version: say "the main versions". No output records the run time; take it from the Kaggle run and write that.

NC-1 OPTIONAL **Two cleaning rules have no decision entry, and one has no count**

owner A (T07)

Found: §3.2 applies two rules without a decision entry: `results.rank = 0 → NaN` (with no count printed) and "race time < laps × 30 s → NaN" (18 rows). Both are explained in the §3.2 markdown, and neither column is a feature; this is about team rule 3 (one decisions.md line per choice) and rule 4 ("count before and after every cleaning step"). **Fix:** print the count before the replace, and add both rules to D2 (or a new D31).

```
print("results.rank = 0 (no fastest-lap rank recorded) set to NaN:", int((df["results"]
["rank"] == 0).sum()))
```

NC-2 OPTIONAL **"Check EVERY conversion" skips the in-place conversions**

owner A

Found: the parse-failure check covers the 8 time conversions. Date conversions and `to_numeric(errors="coerce")` overwrite their column in place, so they are never checked, although the comment says "EVERY" and §3.2 says "every text value converted without failures". **Fix:** either soften the comment and the sentence to "every time conversion", or compare the in-place conversions with the raw text (skip `rank`, whose 0 → NaN is deliberate):

```
inplace = [("races", c) for c in ["date", "fp1_date", "fp2_date", "fp3_date", "quali_date",
"sprint_date"]] + [("drivers", "dob")] \
        + [(t, c) for t in ["results", "sprint_results"] for c in ["number", "position",
"fastestLap", "fastestLapSpeed", "milliseconds"] if c in df[t]]
bad = {f"{t}.{c}": int((~raw[t][c].isin([SENT, ""]) & df[t][c].isna()).sum()) for t, c in
inplace}
assert sum(bad.values()) == 0, bad
```

NC-3 OPTIONAL **Re-running one cell during the evaluation can break later cells**

owners A, C, D

Found: several cells extend `base` by merging. Re-running the §6.5 form cell alone adds the `con_*` columns a second time (pandas makes `_x` / `_y` copies), and §6.6 then fails with a `KeyError`. Part 1's C-8 is the same kind of problem. The brief asks for Run All, which works. **Fix:** in the evaluation, use **Restart & Run All**, never a single cell. For the form cell, drop the columns before merging:

```
CON_FORM_COLS = ["con_podium_rate_last5", "con_avg_finish_last5", "con_mech_rate_last10"]
base = base.drop(columns=CON_FORM_COLS, errors="ignore").merge(con_race[["constructorId",
"raceId"] + CON_FORM_COLS],
                                                              on=["constructorId", "raceId"],
how="left", validate="many_to_one")
```

NC-4 OPTIONAL · SKIP **The missing-package install is not pinned**

owner A

Found: §1 installs `shap`, `lime` and `statsmodels` unpinned when they are missing. **Correction to the first version of this review: leave it.** A pin only applies when the package is missing, so it does not control the versions Kaggle already has installed, and the notebook has already been run and checked on Kaggle.

NC-5 OPTIONAL **One-line hygiene**

- `"Did not start"` in `STATUS_GROUPS` matches no label; delete it *(earlier N4)*.
- `roc_auc_score` is imported twice.
- Q1 prints "so none is a front-row case" as text. Make it `assert miss["grid_clean"].min() > 2`.
- §6.7 and §6.4 type years ("1994–2019", "1950 opener", "before 1958") next to `TRAIN_START` and `TRAIN_END`. Use the constants.
- The weak-feature list in §10 is copied by hand from §6.7.
- In §12, `SHARE_FEATURES` is chosen by substring, and `+ 10` in the qualifying-position check is unexplained. Add a short comment.

NC-6 OPTIONAL **Leave until after the deadline**

These are judgement calls; refactoring now risks new bugs for no change in results:

- three bootstrap implementations (Q1, Q2, §7.1);
- the sprint-weekend rule written three times *(N1)*;
- models picked by label strings such as `startswith("2")` *(N2)*;
- the 152-line Q1 cell and nested conditionals *(N5)*;
- names reused for different things: `train`, `check`, `share` *(N6)*;
- `NAT_TO_COUNTRY` mixing strings and lists;
- the hard-coded American list in §3.6, which repeats D10's map.

CHECKED AND CORRECT **What you can defend with confidence**

- **Leakage controls:** date order is asserted; standings come from `prev_raceId` (asserted strictly earlier); the 314 vs 281 Spa check; `shift(1)` before rolling; team form collapsed per race; the post-race-column guard; the imputer and scaler fitted on train only.
- **Joins:** all 14 primary keys and 5 natural keys are unique, with 0 orphans in 26 foreign-key links; the feature merges use `validate="many_to_one"` / `"one_to_one"`.
- **Grain:** `(raceId, driverId)` is unique (asserted twice); the row funnel 26,759 → 26,668 → 24,715 adds up (–91, –381, –1,365, –207).
- **DE answers:** Q1–Q3 follow from their printed numbers, and the statistics are appropriate: Wilson intervals, chi-square, race-clustered bootstrap, a clustered logit, and stratification for confounding.
- **Brief deliverables:** all five metrics on validation and test, three preprocessing-order experiments (two required), the feature-group ablation, permutation importance and SHAP, SHAP waterfall and LIME, the inference function on a complete and a partial record, and the non-causal statement.
- **Figures 02, 06, 07b and 13b** agree with their text.

8. Order of work for the whole team

□ **Required, text only, no re-run** (≈ 15 min): NS-7 (C), NS-8, NW-1 (C), NW-3 (B), plus Part 1's required items. These 12 are the ones to finish before submitting.

□ **Recommended, text only** (≈ 30 min): NS-3 text route (B), NS-9 (A), NT-1, NT-2, NT-4 to NT-7, NT-9, NT-10, plus Part 1's recommended text items.

□ **Recommended, code** (≈ 1 h, only if there is time for a new run): NS-3 lineage table (B), NS-4 (C), NS-5 (C), plus Part 1's recommended code items.

□ **If any code changes:** one Run All on Kaggle, then re-check every number quoted in the markdown and in DECISIONS.md (the new cells add numbers; NS-3 can change the Q3 answer).

□ **Before committing:** check that the saved notebook still has 129 cells plus the new ones, with no empty cells. A reviewer saw a temporary copy with 129 empty cells inserted; the file on disk is clean (129 cells, none empty, checked again on 15 Oct).

□ **Optional, or after the deadline:** NS-1, NS-2, NS-6, NS-10, NW-2, NT-3, NT-8, NC-1 to NC-3, NC-5, NC-6. **Skip:** NC-4.

How this was checked. **First pass (14 Oct):** all 129 cells, their saved outputs, the 32 figures, DECISIONS.md (D1–D30), README and requirements were read. For Part 1, every number in my cells was recomputed. For each part, two independent reviewers then checked the work: one against the brief, the plan and the dataset notes; one against the engineering principles and the team's working agreements. **Second pass (15 Oct):** every finding was re-checked against `main` at `a2f96ee`, figures 19 and 24, and the brief, and given a priority; four findings were corrected (NT-3, C-1, NW-2, NC-4). The notebook has been run on Kaggle and its quoted numbers checked there. Nothing was re-run for this review because the CSVs are not in the repo; the NS-3 lineage numbers were recomputed by hand from the printed Q3 tables. More detail on Part 1: `Member_D_Verification.md`.